

# SAP Data Replication — Prozessbeschreibung

Vollständige technische Dokumentation des Replikations-Tools  
ABAP-Funktionsbausteine · Python-Client · GUI · Abläufe · Sicherheit

Erstellt: 18.08.2026

## Inhaltsverzeichnis

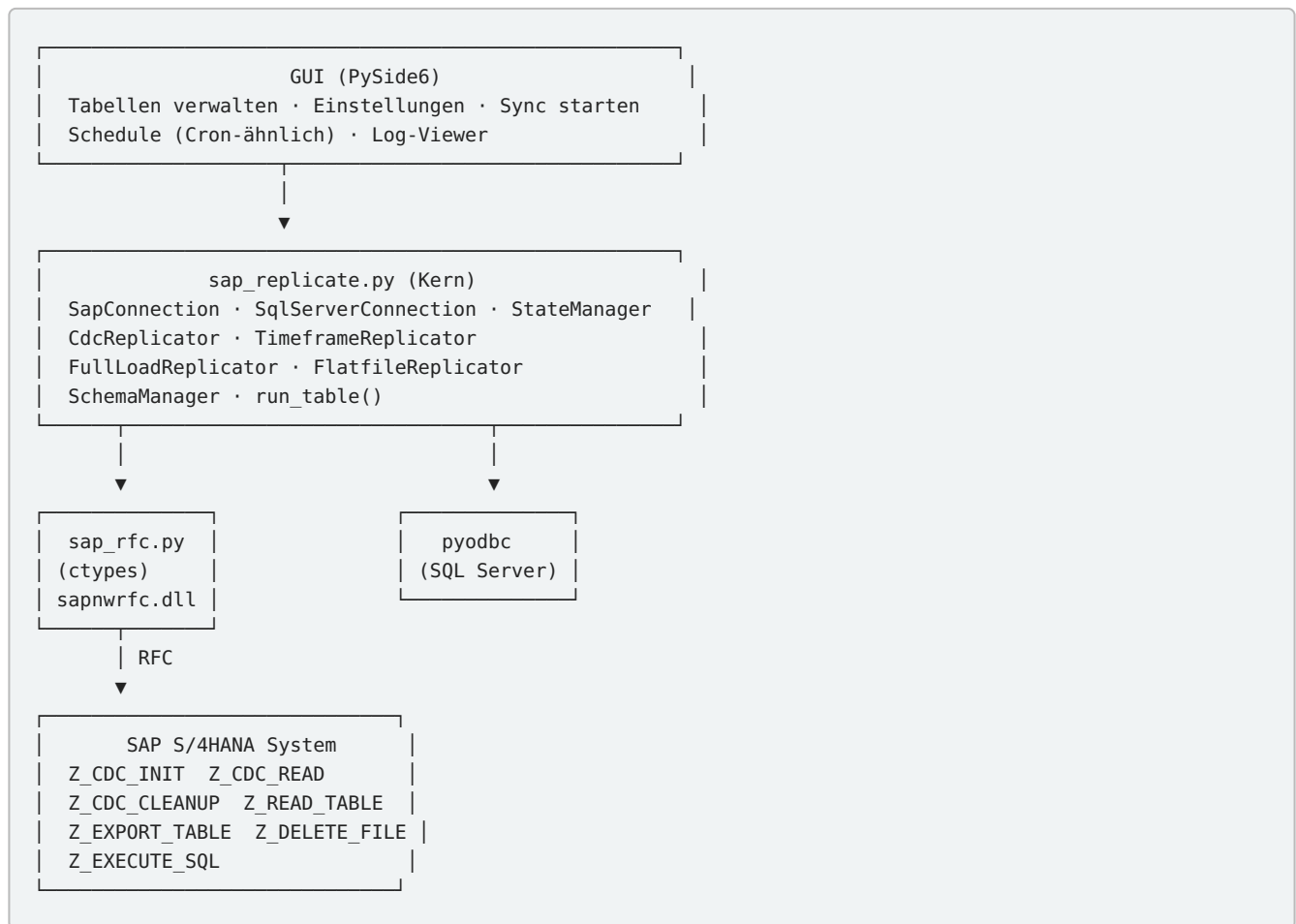
- 1. Überblick — Was macht das Tool?
- 2. Architektur und Komponenten
- 3. GUI — Tabellen hinzufügen und Einstellungen
- 4. Funktionsbausteine — Übersicht und Details
  - 4.1 Z\_CDC\_INIT — CDC initialisieren
  - 4.2 Z\_CDC\_READ — Delta lesen
  - 4.3 Z\_CDC\_CLEANUP — CDC entfernen
  - 4.4 Z\_READ\_TABLE — Tabelle lesen (ODBC-Projekt)
  - 4.5 Z\_EXPORT\_TABLE — CSV-Export
  - 4.6 Z\_DELETE\_FILE — Datei löschen
  - 4.7 Z\_EXECUTE\_SQL — SQL ausführen
- 5. Replikationsmodi im Detail
- 6. Schritt-für-Schritt: Ein kompletter Ablauf
- 7. Sicherheit und Risiken
- 8. Diagnose-Tool

## 1. Überblick — Was macht das Tool?

Das SAP Data Replication Tool repliziert SAP-Tabellen in einen Microsoft SQL Server. Es unterstützt vier Replikationsmodi, die je nach Anforderung kombiniert werden können:

| Modus                     | Beschreibung                                                                                                                       | Funktionsbausteine                    | Trigger nötig? |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|----------------|
| CDC (Change Data Capture) | Trigger-basierte Echtzeit-Delta-Replikation. SAP-Trigger protokollieren jede Änderung (INSERT/UPDATE/DELETE) in einer Log-Tabelle. | Z_CDC_INIT, Z_CDC_READ, Z_CDC_CLEANUP | Ja             |
| Timeframe                 | Zeitfenster-basierte Delta-Replikation ohne Trigger. Lädt alle Datensätze, die in einem Zeitraum geändert wurden.                  | Z_READ_TABLE                          | Nein           |
| Full Load                 | Vollständiger Laden einer Tabelle. TRUNCATE + INSERT aller Daten.                                                                  | Z_READ_TABLE                          | Nein           |
| Flatfile                  | Export als CSV-Datei auf SAP-Server, Download via SCP/SMB, BULK INSERT in SQL Server. Schnellster Modus für große Tabellen.        | Z_EXPORT_TABLE, Z_DELETE_FILE         | Nein           |

## 2. Architektur und Komponenten



**sap\_rfc.py** ist ein ctypes-Wrapper, der die SAP NWRFC SDK DLL direkt aufruft. Er ersetzt die archivierte `pyrfc`-Bibliothek und funktioniert mit Python 3.14. Keine externen Python-Abhängigkeiten außer `ctypes`.

## 3. GUI — Tabellen hinzufügen und Einstellungen

### 3.1 Tabelle hinzufügen

1. Tab "**Tabellen**" → Button "**Hinzufügen**"
2. Dialog öffnet sich mit Feldern:
  - **Tabellenname**: SAP-Tabellenname (z.B. `MARA`)
  - **Zieltabelle**: Name in SQL Server (Standard: gleich wie SAP-Tabelle, ohne Schema-Präfix)
  - **Modus**: Auswahl — `cdc` / `timeframe` / `full` / `flatfile`
  - **Key-Felder**: Komma-separiert (z.B. `MATNR`) — nur für CDC-Modus relevant
  - **Delta-Feld**: Datumsfeld für timeframe-Modus (z.B. `AEDAT`)
  - **Zeitenfenster**: `day` / `week` / `month` / `year` / `all` — nur timeframe-Modus
  - **Chunk-Größe**: Zeilen pro RFC-Aufruf (Standard: 10000)
  - **CSV-Dateipfad**: Verzeichnis auf SAP-Server — nur flatfile-Modus
  - **Felder**: Komma-separiert oder `*` für alle
  - **Aktiv**: Checkbox — nur aktive Tabellen werden synchronisiert
3. Button "**Speichern**" → Tabelle erscheint in der Liste

## 3.2 Einstellungen

Tab "**Einstellungen**" enthält:

- **SAP-Verbindung:** Host, Systemnummer, Mandant, Benutzer, Passwort, Sprache
- **SQL Server:** Connection String (z.B. `DRIVER={ODBC Driver 17 for SQL Server};SERVER=...;DATABASE=...;UID=...;PWD=...`)
- **SSH/SCP:** Host, Benutzer, Key-File, Port — für Flatfile-Download via SCP
- **Test-Buttons:** "SAP testen" und "SQL Server testen" prüfen die Verbindung

## 3.3 Sync starten

Tab "**Ausführen**":

- Button "**Start**" startet die Synchronisation aller aktiven Tabellen in einem Hintergrund-Thread
- Log-Ausgabe erscheint in Echtzeit im Textfeld
- Button "**Stopp**" bricht ab (nach der aktuellen Tabelle)

## 3.4 Schedule

Tab "**Zeitplan**": Schedule-Tabelle anlegen, die automatisch in konfigurierten Intervallen synchronisiert.

## 4. Funktionsbausteine — Übersicht und Details

Das Tool nutzt 7 ABAP-Funktionsbausteine. Die folgenden Abschnitte beschreiben jeden Baustein im Detail: Was er macht, welche Parameter er erwartet, was er technisch in SAP durchführt und welche Risiken bestehen.

| Baustein              | Zweck                                             | Seiteneffekte?                           | Modi                      |
|-----------------------|---------------------------------------------------|------------------------------------------|---------------------------|
| <b>Z_CDC_INIT</b>     | CDC initialisieren: Log-Tabelle + Trigger anlegen | <b>Ja</b> — erstellt Trigger und Tabelle | CDC                       |
| <b>Z_CDC_READ</b>     | Delta aus Log-Tabelle lesen                       | Nein (nur lesen)                         | CDC                       |
| <b>Z_CDC_CLEANUP</b>  | CDC entfernen: Trigger + Log-Tabelle löschen      | <b>Ja</b> — löscht Trigger und Tabelle   | CDC                       |
| <b>Z_READ_TABLE</b>   | Tabelle mit Paging lesen (RFC)                    | Nein                                     | Timeframe, Full Load      |
| <b>Z_EXPORT_TABLE</b> | Tabelle als CSV auf SAP-Server exportieren        | Datei auf SAP-Server                     | Flatfile                  |
| <b>Z_DELETE_FILE</b>  | Datei auf SAP-Server löschen                      | Löscht Datei                             | Flatfile (Cleanup)        |
| <b>Z_EXECUTE_SQL</b>  | Beliebiges SQL auf HANA ausführen                 | Je nach SQL                              | SchemaManager (Metadaten) |

### 4.1 Z\_CDC\_INIT — CDC für eine Tabelle initialisieren

**Aufruf durch:** `CdcReplicator.init_table()` in `sap_replicate.py` (Zeile 586)

#### Parameter

| Parameter              | Typ        | Richtung | Beschreibung                                   |
|------------------------|------------|----------|------------------------------------------------|
| IV_TABLE               | TABNAME    | Import   | SAP-Tabellenname (z.B. MARA )                  |
| IV_KEYFIELDS           | STRING     | Import   | Komma-separierte Schlüsselfelder (z.B. MATNR ) |
| IV_GAP_THRESHOLD_HOURS | I          | Import   | Lückenschwelle in Stunden (Default 24)         |
| EV_LOG_TABLE           | TABNAME    | Export   | Name der erstellten Log-Tabelle                |
| EV_TRIGGER_EXISTS      | CHAR1      | Export   | 'X' = Trigger existierte bereits               |
| EV_GAP_DETECTED        | CHAR1      | Export   | 'X' = Lücke erkannt (Trigger war weg)          |
| EV_LAST_LOG_SEQ        | I          | Export   | Letzte SEQ in Log-Tabelle                      |
| EV_LAST_LOG_TIME       | TIMESTAMPL | Export   | Zeitstempel des letzten Log-Eintrags           |
| EV_ERROR               | STRING     | Export   | Fehlermeldung (leer = OK)                      |

#### Was der Baustein technisch in SAP durchführt

- Validierung:** Tabellenname wird auf alphanumerische Zeichen geprüft (kein SQL-Injection möglich)
- Namensgenerierung:**
  - Log-Tabelle: `Z_<Tabelle>_CDC_LOG` (z.B. `Z_MARA_CDC_LOG` )
  - Trigger: `Z_<Tabelle>_CDC_TRG_INS/UPD/DEL`
  - Sequence: `Z_<Tabelle>_CDC_SEQ`
  - Bei Tabellennamen > 18 Zeichen: Hash-basierter Kurzname (6 Zeichen) um HANA-Limit von 32 Zeichen einzuhalten
- Prüfen ob Log-Tabelle existiert:** `SELECT COUNT(*) FROM (lv_log_table)` — dynamisches Open SQL
- Log-Tabelle erstellen (falls nicht vorhanden):**

```
CREATE COLUMN TABLE Z_MARA_CDC_LOG (
  SEQ INTEGER NOT NULL,
  OPERATION VARCHAR(1) NOT NULL,
  KEYVALUES NVARCHAR(5000),
  Timestmp TIMESTAMP NOT NULL,
  PRIMARY KEY (SEQ)
)
```

Dies ist eine HANA-spezifische DDL, ausgeführt via ADBC ( `cl_sql_statement->execute_ddl()` ).

#### 5. Sequence erstellen (falls nicht vorhanden):

```
CREATE SEQUENCE Z_MARA_CDC_SEQ
  START WITH 1 INCREMENT BY 1 NO CACHE
```

6. **Lücken-Erkennung:** Wenn die Log-Tabelle Einträge hat aber kein Trigger existiert, und der letzte Eintrag älter als 24 Stunden ist → `EV_GAP_DETECTED = 'X'`

7. **Prüfen ob Trigger existiert:** `SELECT COUNT(*) FROM SYS.TRIGGERS WHERE TRIGGER_NAME = 'Z_MARA_CDC_TRG_INS'`

8. **Wenn Trigger bereits existiert:** `EV_TRIGGER_EXISTS = 'X'` , Funktion beendet (idempotent — mehrfach aufrufbar ohne Duplikate)

9. **Trigger erstellen (falls nicht vorhanden):** Drei Trigger via HANA DDL:

```
-- INSERT Trigger
CREATE TRIGGER Z_MARA_CDC_TRG_INS
AFTER INSERT ON MARA
REFERENCING NEW ROW AS new_row
FOR EACH ROW
BEGIN
  INSERT INTO Z_MARA_CDC_LOG
    (SEQ, OPERATION, KEYVALUES, Timestmp)
  VALUES (Z_MARA_CDC_SEQ.NEXTVAL, 'I',
    :new_row.MATNR, CURRENT_TIMESTAMP);
END

-- UPDATE Trigger (identisch, OPERATION = 'U')
-- DELETE Trigger (REFERENCING OLD ROW, OPERATION = 'D')
```

#### Was passiert in SAP:

- Eine neue HANA-Tabelle ( `Z_<Tabelle>_CDC_LOG` ) wird erstellt
- Eine HANA-Sequence ( `Z_<Tabelle>_CDC_SEQ` ) wird erstellt
- Drei HANA-Trigger werden auf der Quelltable erstellt (INSERT, UPDATE, DELETE)
- **Ab diesem Moment protokollieren die Trigger jede Änderung an der Tabelle**
- Die Trigger schreiben nur Schlüsselfelder + Operationstyp in die Log-Tabelle (nicht die gesamten Daten)
- Die Trigger verwenden `:new_row.<Feldname>` — für Composite Keys werden die Felder mit `|` verkettet

**Idempotent:** Der Baustein kann beliebig oft aufgerufen werden. Wenn Trigger und Log-Tabelle bereits existieren, passiert nichts — er meldet nur `EV_TRIGGER_EXISTS = 'X'` .

## 4.2 Z\_CDC\_READ — Delta aus Log-Tabelle lesen

**Aufruf durch:** `CdcReplicator.read_delta()` in `sap_replicate.py` (Zeile 610)

### Parameter

| Parameter     | Typ             | Richtung | Beschreibung                                                 |
|---------------|-----------------|----------|--------------------------------------------------------------|
| IV_TABLE      | TABNAME         | Import   | SAP-Tabellenname                                             |
| IV_FROM_SEQ   | I               | Import   | Ab dieser SEQ lesen (Lese-Pointer)                           |
| IV_CHUNK_SIZE | I               | Import   | Max. Zeilen pro Aufruf (Default 10000)                       |
| EV_ROW_COUNT  | I               | Export   | Tatsächlich zurückgegebene Zeilen                            |
| EV_NEXT_SEQ   | I               | Export   | Nächste zu lesende SEQ                                       |
| EV_HAS_MORE   | CHAR1           | Export   | 'X' = weitere Daten verfügbar                                |
| EV_ERROR      | STRING          | Export   | Fehlermeldung                                                |
| ET_FIELDS     | LIKE ZSQL_FIELD | Tables   | Spaltenmetadaten (Feldname, Typ, Länge, Dezimalen)           |
| ET_DATA       | LIKE ZSQL_ROW   | Tables   | Pipe-delimited Daten: <code>OPERATION feld1 feld2 ...</code> |

### Was der Baustein technisch in SAP durchführt

#### 1. Log-Tabelle lesen via ADBC:

```
SELECT SEQ, OPERATION, KEYVALUES, TIMESTMP
FROM Z_MARA_CDC_LOG
WHERE SEQ > <iv_from_seq>
ORDER BY SEQ ASC
```

Verwendet `cl_sql_connection`, `cl_sql_statement`, `cl_sql_result_set`.

#### 2. Für jeden Log-Eintrag:

- **DELETE (Operation 'D'):** Nur Schlüsselfelder aus KEYVALUES werden als Pipe-delimited Row übernommen. Kein Zugriff auf die Originaltabelle.
- **INSERT/UPDATE (Operation 'I'/'U'):** Schlüsselfelder aus KEYVALUES werden per `SPLIT AT '|'` zerlegt. Eine `WHERE`-Klausel wird gebaut (`MATNR = '0000000001'`). Dann `SELECT SINGLE * FROM (Tabelle) WHERE (lv_where)` — dynamisches Open SQL liest die komplette Zeile.

#### 3. Typkonvertierung für MSSQL:

- **DATE** (YYYYMMDD → YYYY-MM-DD)
- **TIME** (HHMMSS → HH:MM:SS)
- **PACKED** → Dezimalpunkt, keine Tausendertrennzeichen
- **FLOAT** → wie PACKED
- **RAW/HEX** → `0x`-Präfix für MSSQL VARBINARY
- **CHAR** → Trailing Spaces entfernt
- **INT** → Klartext-Zahl

- 4. **Chunking:** Nach `IV_CHUNK_SIZE` Zeilen wird abgebrochen. `EV_NEXT_SEQ` gibt die nächste SEQ, `EV_HAS_MORE` zeigt ob weitere Daten vorhanden sind (via `SELECT COUNT(*)` auf Log-Tabelle).

**Keine Seiteneffekte:** Z\_CDC\_READ liest nur Daten. Es verändert weder die Log-Tabelle noch die Quelltable. Die Log-Einträge bleiben erhalten bis Z\_CDC\_CLEANUP aufgerufen wird.

### 4.3 Z\_CDC\_CLEANUP — CDC entfernen

**Aufruf durch:** `CdcReplicator.cleanup()` in `sap_replicate.py` (Zeile 758, 770)

#### Parameter

| Parameter     | Typ     | Richtung | Beschreibung                                                      |
|---------------|---------|----------|-------------------------------------------------------------------|
| IV_TABLE      | TABNAME | Import   | SAP-Tabellenname                                                  |
| IV_UP_TO_SEQ  | I       | Import   | Log bis zu dieser SEQ löschen (Modus 1)                           |
| IV_REMOVE_ALL | CHAR1   | Import   | 'X' = alles entfernen: Trigger + Sequence + Log-Tabelle (Modus 2) |
| EV_DELETED    | I       | Export   | Anzahl gelöschter Log-Einträge                                    |
| EV_ERROR      | STRING  | Export   | Fehlermeldung                                                     |

#### Modus 1: Log-Einträge aufräumen (IV\_UP\_TO\_SEQ > 0)

Nach erfolgreicher Synchronisation werden alte Log-Einträge gelöscht:

```
DELETE FROM Z_MARA_CDC_LOG WHERE SEQ <= <iv_up_to_seq>
```

Die Trigger und Sequence bleiben erhalten. Nur die bereits synchronisierten Einträge werden gelöscht.

#### Modus 2: Komplett entfernen (IV\_REMOVE\_ALL = 'X')

Entfernt alle Komponenten des CDC in folgender Reihenfolge:

1. DROP TRIGGER Z\_MARA\_CDC\_TRG\_INS
2. DROP TRIGGER Z\_MARA\_CDC\_TRG\_UPD
3. DROP TRIGGER Z\_MARA\_CDC\_TRG\_DEL
4. DROP SEQUENCE Z\_MARA\_CDC\_SEQ
5. DROP TABLE Z\_MARA\_CDC\_LOG

**Achtung:** Modus 2 entfernt die Trigger komplett. Ab diesem Moment werden keine Änderungen mehr protokolliert. Eine neue CDC-Initialisierung (Z\_CDC\_INIT) ist nötig, um CDC wieder zu aktivieren.

#### 4.4 Z\_READ\_TABLE — Tabelle mit Paging lesen

**Herkunft:** ODBC-Projekt (sap-odbc-abap) — wird per Querverweis referenziert, nicht im Replication-Repo dupliziert

**Aufruf durch:** TimeframeReplicator.sync\_table() (Zeile 887), FullLoadReplicator.sync\_table() (Zeile 983)

##### Parameter

| Parameter   | Typ             | Richtung | Beschreibung                                        |
|-------------|-----------------|----------|-----------------------------------------------------|
| IV_TABLE    | TABNAME         | Import   | SAP-Tabellenname                                    |
| IV_FIELDS   | STRING          | Import   | Feldliste komma-separiert, * = alle                 |
| IV_WHERE    | STRING          | Import   | WHERE-Klausel (optional, z.B. AEDAT >= '20240101' ) |
| IV_ORDERBY  | STRING          | Import   | ORDER BY (optional)                                 |
| IV_ROWSKIPS | I               | Import   | Anzahl zu überspringende Zeilen (Paging)            |
| IV_ROWCOUNT | I               | Import   | Max. Zeilen pro Aufruf                              |
| EV_ERROR    | STRING          | Export   | Fehlermeldung                                       |
| ET_DATA     | LIKE ZSQL_ROW   | Tables   | Pipe-delimited Daten: feld1 feld2 ...               |
| ET_FIELDS   | LIKE ZSQL_FIELD | Tables   | Spaltenmetadaten                                    |

##### Was der Baustein technisch in SAP durchführt

1. **Dynamisches SELECT:** SELECT (felder) FROM (tabelle) WHERE (where) ORDER BY (orderby) — vollständig dynamisches Open SQL
2. **Paging:** UP TO IV\_ROWCOUNT ROWS + SKIP IV\_ROWSKIPS ROWS (Offset-based)
3. **Typkonvertierung:** Gleiche Konvertierungen wie Z\_CDC\_READ (DATE, TIME, PACKED, FLOAT, RAW, CHAR, INT)
4. **Rückgabe:** Pipe-delimited Rows in ET\_DATA, Metadaten in ET\_FIELDS

**Keine Seiteneffekte:** Z\_READ\_TABLE ist 100% lesend. Keine Trigger, keine Tabellen, keine Dateien.

#### 4.5 Z\_EXPORT\_TABLE — Tabelle als CSV-Flatfile exportieren

**Aufruf durch:** FlatfileReplicator.sync\_table() in sap\_replicate.py (Zeile 1338)

##### Parameter

| Parameter     | Typ     | Richtung | Beschreibung                              |
|---------------|---------|----------|-------------------------------------------|
| IV_TABLE      | TABNAME | Import   | SAP-Tabellenname                          |
| IV_DATE_FIELD | STRING  | Import   | Datumfeld für Filter (optional)           |
| IV_DATE_FROM  | DATUM   | Import   | Von-Datum YYYYMMDD (optional)             |
| IV_DATE_TO    | DATUM   | Import   | Bis-Datum YYYYMMDD (optional)             |
| IV_FIELDS     | STRING  | Import   | Feldliste, * = alle                       |
| IV_MAX_ROWS   | I       | Import   | Max. Zeilen (0 = alle)                    |
| IV_FILE_PATH  | STRING  | Import   | Verzeichnis für CSV (z.B. /usr/sap/tmp/ ) |
| EV_FILE_NAME  | STRING  | Export   | Vollständiger Dateipfad der erzeugten CSV |



|              |        |        |                             |
|--------------|--------|--------|-----------------------------|
| EV_ROW_COUNT | I      | Export | Anzahl geschriebener Zeilen |
| EV_FILE_SIZE | I      | Export | Dateigröße in Bytes         |
| EV_ERROR     | STRING | Export | Fehlermeldung               |

#### Was der Baustein technisch in SAP durchführt

1. **Tabellenname validieren** (alphanumerisch, kein SQL-Injection)
2. **DDIF\_NAMETAB\_GET** aufrufen um Feldmetadaten zu erhalten (Typ, Länge, Dezimalen)
3. **Dateipfad bauen:** /usr/sap/tmp/MARA\_20240115\_143022.csv (Tabelle + Datum + Uhrzeit)
4. **Datei öffnen:** OPEN DATASET <datei> FOR OUTPUT IN TEXT MODE ENCODING DEFAULT
5. **CSV-Header schreiben:** Feldnamen pipe-delimited (z.B. MATNR|MTART|MATKL )
6. **Daten in Blöcken von 50.000 Zeilen lesen:**
  - Dynamisches SELECT (felder) FROM (tabelle) WHERE (where) ORDER BY (orderby) UP TO 50000 ROWS
  - **Keyset-Paging:** Nach jedem Block wird der letzte Primärschlüssel gemerkt. Der nächste Block liest WHERE key > 'letzter\_key' . Dadurch wird der SAP-Speicher nicht überlastet.
  - Typkonvertierung: DATE → YYYY-MM-DD, TIME → HH:MM:SS, PACKED → Dezimalpunkt, etc.
  - Jede Zeile wird mit TRANSFER lv\_row TO <datei> in die CSV geschrieben
7. **Datei schließen:** CLOSE DATASET <datei>

**Seiteneffekt:** Erstellt eine CSV-Datei auf dem SAP-Server-Dateisystem. Die Datei wird nach erfolgreichem Download und Import durch Z\_DELETE\_FILE wieder gelöscht.

## 4.6 Z\_DELETE\_FILE — Datei auf SAP-Server löschen

**Aufruf durch:** FlatfileReplicator.sync\_table() in sap\_replicate.py (Zeile 1355, 1380)

### Parameter

| Parameter    | Typ    | Richtung | Beschreibung            |
|--------------|--------|----------|-------------------------|
| IV_FILE_PATH | STRING | Import   | Vollständiger Dateipfad |
| EV_ERROR     | STRING | Export   | Fehlermeldung           |

### Was der Baustein technisch in SAP durchführt

- DELETE DATASET <datei>
- Fehler werden abgefangen und in EV\_ERROR gemeldet

**Nur für Flatfile-Modus:** Löscht die temporäre CSV-Datei nach erfolgreichem Download. Wird nur mit Pfaden aufgerufen, die von Z\_EXPORT\_TABLE erzeugt wurden.

## 4.7 Z\_EXECUTE\_SQL — Beliebiges SQL auf HANA ausführen

**Aufruf durch:** SchemaManager.get\_table\_fields() (Zeile 298), get\_indexes() (Zeile 327), create\_table() (Zeile 355)

### Parameter

### Richtung

| Parameter   | Typ             |        | Beschreibung                |
|-------------|-----------------|--------|-----------------------------|
| IV_SQL      | STRING          | Import | Vollständiges SQL-Statement |
| IV_MAX_ROWS | I               | Import | Max. Zeilen (Default 30000) |
| EV_ERROR    | STRING          | Export | Fehlermeldung               |
| ET_DATA     | LIKE ZSQL_ROW   | Tables | Ergebnis-Daten              |
| ET_FIELDS   | LIKE ZSQL_FIELD | Tables | Spaltenmetadaten            |

### Verwendung im Tool

Wird ausschließlich vom SchemaManager verwendet um:

- Felddefinitionen aus DD03L zu lesen ( SELECT FIELDNAME, INTTYPE, INTLEN, DECIMALS, KEYFLAG FROM DD03L WHERE TABNAME = 'MARA' )
- Indexdefinitionen aus DD12L/DD17S zu lesen
- Tabellen im SQL Server anzulegen (Schema-Synchronisation)

**Keine schädlichen SQL-Statements:** Das Tool verwendet Z\_EXECUTE\_SQL nur für SELECT -Abfragen auf SAP-DDIC-Tabellen (DD03L, DD12L, DD17S). Keine INSERT , UPDATE , DELETE oder DROP über Z\_EXECUTE\_SQL.

## 5. Replikationsmodi im Detail

### 5.1 CDC-Modus (Change Data Capture)

1. `Z_CDC_INIT(IV_TABLE='MARA', IV_KEYFIELDS='MATNR')`
  - Erstellt Log-Tabelle + 3 Trigger auf MARA
  - Trigger protokollieren jede Änderung
2. Schleife (chunk-weise):
  - `Z_CDC_READ(IV_TABLE='MARA', IV_FROM_SEQ=0, IV_CHUNK_SIZE=10000)`
  - Liest Delta aus Log-Tabelle
  - Für INSERT/UPDATE: liest Originalzeile via `SELECT SINGLE *`
  - Für DELETE: nur Schlüsselfelder
  - Gibt `ET_DATA` zurück (pipe-delimited, OPERATION-Prefix)
  
  - Python: `MERGE INTO dbo.MARA (UPSERT) oder DELETE`
  - `EV_NEXT_SEQ` merken, `EV_HAS_MORE` prüfen
  - Wenn `has_more='X'`: nächste Chunk lesen
3. `Z_CDC_CLEANUP(IV_TABLE='MARA', IV_UP_TO_SEQ=<last_seq>)`
  - Löscht bereits synchronisierte Log-Einträge
  - Trigger bleiben aktiv für weitere Änderungen

### 5.2 Timeframe-Modus (trigger-frei)

1. Zeitfenster berechnen:
  - `window='month'` → aktueller Monat + Vormonat
  - `window='week'` → aktuelle Woche + Vorwoche
  - `window='day'` → heute + gestern
  - `window='all'` → keine Filter (Full-Load)
2. Wenn `window='all'`:
  - `TRUNCATE TABLE dbo.[MARA]`
  - Löscht alle Daten im SQL Server
3. Sonst:
  - `DELETE FROM dbo.[MARA] WHERE AEDAT >= '2024-01-01'`
  - Löscht nur den Zeitraum im SQL Server
4. Schleife (chunk-weise):
  - `Z_READ_TABLE(IV_TABLE='MARA', IV_FIELDS='*',`  
`IV_WHERE='AEDAT >= ''20240101''',`  
`IV_ROWSKIPS=0, IV_ROWCOUNT=10000)`
  - Liest Daten von SAP mit Paging
  - Python: `INSERT INTO dbo.MARA`
5. Nächster Chunk:
  - `IV_ROWSKIPS += 10000` (Offset-basiert)

### 5.3 Full-Load-Modus

1. `TRUNCATE TABLE dbo.[MARA]`
  - Löscht alle Daten im SQL Server
2. Schleife (chunk-weise):

```
Z_READ_TABLE(IV_TABLE='MARA', IV_FIELDS='*',  
              IV_ROWSKIPS=0, IV_ROWCOUNT=10000)  
→ Liest alle Daten von SAP
```

3. Nächster Chunk:  
IV\_ROWSKIPS += 10000

4. Bei EV\_HAS\_MORE != 'X': fertig

## 5.4 Flatfile-Modus (schnellster für große Tabellen)

1. Z\_EXPORT\_TABLE(IV\_TABLE='MARA', IV\_FIELDS='\*',  
 IV\_FILE\_PATH='/usr/sap/tmp/')  
→ SAP exportiert MARA als CSV auf Server-Dateisystem  
→ Datei: /usr/sap/tmp/MARA\_20240115\_143022.csv  
→ Pipe-delimited mit Header-Zeile
2. Download der Datei:  
→ SCP (SSH): scp user@sap:/usr/sap/tmp/MARA\_\*.csv /tmp/  
→ SMB (Windows): \sap-server\sap mp\MARA\_\*.csv  
→ Local (gemountet): direkter Zugriff
3. BULK INSERT in SQL Server:  
BULK INSERT dbo.[MARA] FROM 'C: emp\MARA\_\*.csv'  
WITH (FORMAT='CSV', FIELDTERMINATOR='|',  
 FIRSTROW=2, TABLOCK)
4. Bei Fehlern: Batch-Insert via executemany()
5. Z\_DELETE\_FILE(IV\_FILE\_PATH='/usr/sap/tmp/MARA\_20240115\_143022.csv')  
→ Löscht die temporäre CSV-Datei auf SAP-Server

## 6. Schritt-für-Schritt: Ein kompletter Ablauf

Beispiel: Erste Synchronisation der Tabelle MARA im CDC-Modus.

### Schritt 1: Vorbereitung

1. GUI starten via `start.bat`
2. Tab **"Einstellungen"**: SAP-Verbindung und SQL Server konfigurieren
3. **"SAP testen"** und **"SQL Server testen"** klicken

### Schritt 2: Diagnose (empfohlen vor erstem Sync)

```
python sap_diagnose.py --host ... --sysnr ... --client ... --user ... --password ...
```

- Prüft ob alle Funktionsbausteine existieren
- Testet `Z_READ_TABLE` (1 Zeile aus `ZTLANGTXT`)
- Testet `Z_EXPORT_TABLE` + `Z_DELETE_FILE` (1 Zeile + Cleanup)
- Keine Seiteneffekte, 100% sicher

### Schritt 3: Tabelle hinzufügen

1. Tab **"Tabellen"** → **"Hinzufügen"**
2. Tabellename: `MARA`
3. Modus: `cdc`
4. Key-Felder: `MATNR`
5. Aktiv: ✓
6. **"Speichern"**

### Schritt 4: Synchronisation starten

1. Tab **"Ausführen"** → **"Start"**

### Was passiert jetzt intern:

| Schritt | Funktionsbaustein          | Aktion in SAP                                                          | Aktion in SQL Server                                                         |
|---------|----------------------------|------------------------------------------------------------------------|------------------------------------------------------------------------------|
| 1       | SchemaManager              | <code>Z_EXECUTE_SQL</code> : DD03L lesen (Felddefinitionen MARA)       | CREATE TABLE dbo.MARA (falls nicht vorhanden)                                |
| 2       | <code>Z_CDC_INIT</code>    | Log-Tabelle <code>Z_MARA_CDC_LOG</code> + 3 Trigger auf MARA erstellen | —                                                                            |
| 3       | <code>Z_CDC_READ</code>    | Delta aus Log-Tabelle lesen ( <code>SEQ &gt; 0</code> , 10000 Zeilen)  | —                                                                            |
| 4       | —                          | —                                                                      | MERGE INTO dbo.MARA (UPSERT) oder DELETE                                     |
| 5       | <code>Z_CDC_READ</code>    | Nächste Chunk ( <code>SEQ &gt; last_seq</code> )                       | MERGE / DELETE                                                               |
| 6       | <code>Z_CDC_CLEANUP</code> | Log-Einträge bis <code>last_seq</code> löschen                         | —                                                                            |
| 7       | StateManager               | —                                                                      | <code>replication_state</code> aktualisieren ( <code>last_seq</code> merken) |

## Schritt 5: Nächste Synchronisation (inkrementell)

Bei der nächsten Ausführung beginnt Z\_CDC\_READ bei der SEQ, die im State gespeichert wurde. Nur neue Änderungen seit der letzten Synchronisation werden gelesen.

## 7. Sicherheit und Risiken

### 7.1 Was sicher ist (keine Seiteneffekte)

| Aktion                            | Baustein                         | Risiko                              |
|-----------------------------------|----------------------------------|-------------------------------------|
| SAP-Verbindung testen             | — (Login only)                   | Keines                              |
| Funktionsbaustein-Existenz prüfen | — (Metadaten)                    | Keines                              |
| Tabelle lesen                     | Z_READ_TABLE                     | Keines (nur SELECT)                 |
| Delta lesen                       | Z_CDC_READ                       | Keines (nur SELECT auf Log-Tabelle) |
| Schema lesen                      | Z_EXECUTE_SQL (SELECT auf DD03L) | Keines (nur SELECT)                 |
| Diagnose ohne --init-test         | Alle (nur Lesen)                 | Keines                              |

### 7.2 Was Seiteneffekte hat

| Aktion             | Baustein                | Was wird erstellt/verändert                     | Rückgängig?                               |
|--------------------|-------------------------|-------------------------------------------------|-------------------------------------------|
| CDC initialisieren | Z_CDC_INIT              | Log-Tabelle, Sequence, 3 Trigger auf Quelltable | Ja, via Z_CDC_CLEANUP (IV_REMOVE_ALL='X') |
| Log aufräumen      | Z_CDC_CLEANUP (Modus 1) | Löscht Log-Einträge bis SEQ                     | Nein (Daten bereits synchronisiert)       |
| CDC entfernen      | Z_CDC_CLEANUP (Modus 2) | DROP Trigger, Sequence, Log-Tabelle             | Nein, aber Z_CDC_INIT kann neu starten    |
| CSV exportieren    | Z_EXPORT_TABLE          | CSV-Datei auf SAP-Server                        | Ja, via Z_DELETE_FILE                     |
| Datei löschen      | Z_DELETE_FILE           | Löscht Datei                                    | Nein                                      |

### 7.3 SQL-Injection-Schutz

Alle Bausteine validieren Tabellen- und Feldnamen auf alphanumerische Zeichen ( A-Z, 0-9, \_, / ). Dynamische SQL-Statements verwenden nur validierte Namen. Schlüsselwerte in WHERE-Klauseln werden mit Single-Quote-Escaping ( ' ' → ' ' ' ' ) abgesichert.

### 7.4 Trigger-Sicherheit

Die Trigger protokollieren nur Schlüsselfelder + Operationstyp. Sie schreiben keine sensiblen Daten. Die Trigger verwenden `AFTER INSERT/UPDATE/DELETE` — sie feuern *nach* der eigentlichen Operation und können diese nicht blockieren. Die Trigger-Bedingung ist minimal und hat vernachlässigbaren Performance-Einfluss auf die Quelltable.

### 7.5 Was passieren kann, wenn ein Trigger fehlerhaft ist

Wenn ein Trigger einen Fehler wirft (z.B. weil die Log-Tabelle gelöscht wurde während eine Transaktion läuft), kann die ursprüngliche SAP-Transaktion fehlschlagen. Dies ist ein theoretisches Risiko bei allen Trigger-basierten CDC-Lösungen. Der Baustein Z\_CDC\_CLEANUP mit `IV_REMOVE_ALL='X'` kann verwendet werden, um die Trigger zu entfernen und die Quelltable wieder in den Normalzustand zu versetzen.

## 8. Diagnose-Tool (sap\_diagnose.py)

Das Diagnose-Tool prüft alle Funktionsbausteine **ohne Seiteneffekte**:

```
python sap_diagnose.py --host sap.example.com --sysnr 00 --client 100 --user MYUSER --password MYPASS
```

Ohne `--init-test` macht das Tool **nur Lesen**:

- Verbindung testen
- Existenz aller 7 Funktionsbausteine prüfen (nur Metadaten)
- `Z_READ_TABLE`: 1 Zeile aus `ZTLANGTXT` lesen
- `Z_EXPORT_TABLE`: 1 Zeile nach `/tmp/` exportieren + sofort löschen
- `Z_EXECUTE_SQL`: harmloses `SELECT`
- CDC-Status: prüfen ob Log-Tabellen existieren (nur lesend)
- Zusammenfassung mit Ampel-Status

Mit `--init-test` (opt-in, mit Warnung):

- Vollständiger CDC-Lebenszyklus auf `ZTLANGTXT` (kleine, unkritische Tabelle)
- `Z_CDC_INIT` → `Z_CDC_READ` → `Z_CDC_CLEANUP` (`IV_REMOVE_ALL='X'`)
- Am Ende wird alles wieder entfernt

**Hinweis:** Die Diagnose-Funktion soll in die GUI integriert werden (geplant). Aktuell ist sie als Kommandozeilen-Tool verfügbar.